

快速上手

# Kimi K2.6

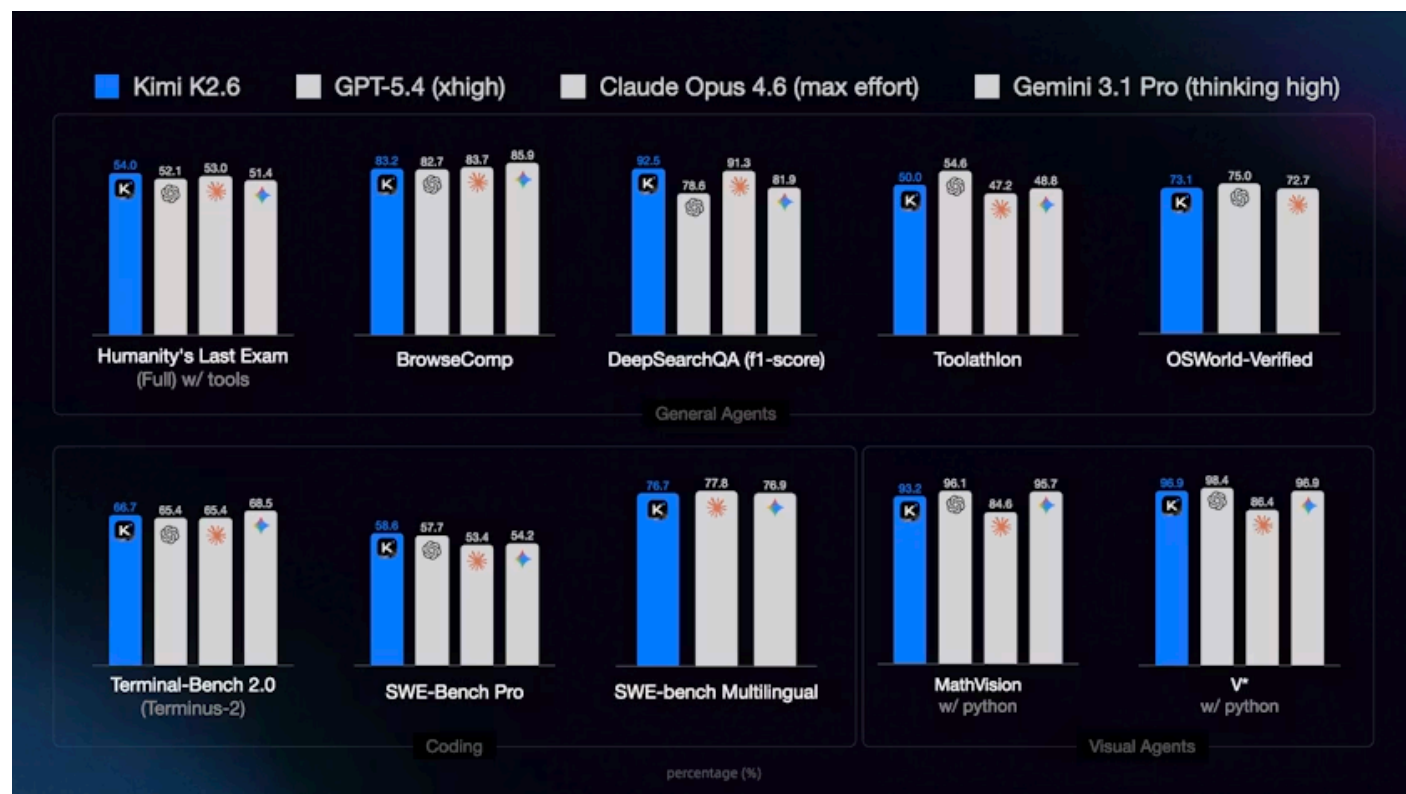
📄 复制页面



## Kimi K2.6 模型介绍

Kimi K2.6 是 Kimi 最新最智能的模型，Kimi K2.6 的通用 Agent、代码、视觉理解等综合能力得到全面提升，其中在博士级难度的完整版人类最后的考试（Humanity's Last Exam）、在考察模型真实软件工程能力的 SWE-Bench Pro、评估 Agent 深度检索能力的 DeepSearchQA 等基准测试中均取得行业领先的成绩，同时支持文本、图片与视频输入，思考与非思考模式，对话与 Agent 任务。

### 技术Blog



## 长程编码能力突破

## 超长上下文支持

- `kimi-k2.6`、`kimi-k2.5`、`kimi-k2-0905-preview`、`kimi-k2-turbo-preview`、`kimi-k2-thinking`、`kimi-k2-thinking-turbo` 模型均提供 256K 上下文窗口

## 长思考能力

- Kimi K2.6 仍然具备超强的思考能力，支持多步工具调用和推理，擅长解决复杂问题，如复杂的逻辑推理、数学问题、代码编写等。

## 立即开始

- **立即体验**：在开发工作台，快速通过交互式操作测试模型在业务场景上的效果
- **申请 API Key**：立即通过 API 调用测试

## 调用示例

以下是完整的调用示例，帮助您快速上手 Kimi K2.6 多模态模型。

## 安装 OpenAI SDK

Kimi API 完全兼容 OpenAI 的 API 格式，你可以通过如下方式来安装 OpenAI SDK：

```
pip install --upgrade 'openai>=1.0'
```

## 验证安装结果

🎉 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

**KIMI** 开放平台

可能是 `OpenAI version = 1.10.0`，表示 OpenAI SDK 已经安装成功，当前 python 实际使用了 OpenAI

## 图片理解代码示例

```
import base64
```

**KIMI** 开放平台



```
from openai import OpenAI
```

```
client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url="https://api.moonshot.cn/v1",
)
```

```
# 在这里，你需要将 kimi.png 文件替换为你想让 Kimi 识别的图片的地址
image_path = "kimi.png"
```

```
with open(image_path, "rb") as f:
    image_data = f.read()
```

```
# 我们使用标准库 base64.b64encode 函数将图片编码成 base64 格式的 image_url
image_url = f"data:image/{os.path.splitext(image_path)[1].lstrip('.')};base64,{bas
```

```
completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {"role": "system", "content": "你是 Kimi。"},
        {
            "role": "user",
            # 注意这里，content 由原来的 str 类型变更为一个 list，这个 list 中包含多个部分
            # 文字 (text) 是一个部分 (part)
            "content": [
                {
                    "type": "image_url", # <-- 使用 image_url 类型来上传图片，内容为使
                    "image_url": {
                        "url": image_url,
                    },
                },
                {
                    "type": "text",
                    "text": "请描述图片的内容。", # <-- 使用 text 类型来提供文字指令，例如
                },
            ],
        },
    ],
)
```

**KIMI** 开放平台



```
print(completion.choices[0].message.content)
```

>

## 视频理解代码示例

**KIMI** 开放平台



```
completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {"role": "system", "content": "你是 Kimi。"},
        {
            "role": "user",
            # 注意这里, content 由原来的 str 类型变更为一个 list, 这个 list 中包含多个部分
            # 文字 (text) 是一个部分 (part)
            "content": [
                {
                    "type": "video_url", # <-- 使用 video_url 类型来上传视频, 内容为使
                    "video_url": {
                        "url": video_url,
                    },
                },
                {
                    "type": "text",
                    "text": "请描述视频的内容。", # <-- 使用 text 类型来提供文字指令, 例如
                },
            ],
        },
    ],
)
```

**KIMI** 开放平台



```
print(completion.choices[0].message.content)
```

>

## 多模态工具能力示例

Kimi K2.6 模型综合了多种能力。以下是一个展示 K2.6 视觉理解+工具调用能力的示例。

首先将这个示例视频下载到本地，比如 `/path/to/test_video.mp4`



然后运行以下代码

**KIMI** 开放平台

```
import json
import subprocess
import tempfile
from pathlib import Path
from openai import OpenAI

tools = [{
    "type": "function",
    "function": {
        "name": "watch_video_clip",
        "description": "Watch a video file or a sub-clip of it. If start_time and",
        "parameters": {
            "type": "object",
            "properties": {
                "path": {
                    "type": "string",
                    "description": "The path to the video file to watch"
                },
                "start_time": {
                    "type": "number",
                    "description": "The start time of the clip in seconds (optional",
                },
                "end_time": {
                    "type": "number",
                    "description": "The end time of the clip in seconds (optional",
                }
            },
            "required": ["path"]
        }
    }
}]

def watch_video_clip(path: str, start_time: float | None = None, end_time: float
```

"""

Watch a video file or a sub-clip of it.

Args:

path: The path to the video file to watch



**KIMI** 开放平台



Returns:

A list of content blocks in MultiModal Tool API format

```
""" >
```

```
video_path = Path(path)
if not video_path.exists():
    raise FileNotFoundError(f"Video file not found: {path}")

# Get video duration if needed
if start_time is None and end_time is None:
    # Return entire video
    with open(path, "rb") as f:
        video_base64 = base64.b64encode(f.read()).decode("utf-8")
    return [
        {"type": "video_url", "video_url": {"url": f"data:video/mp4;base64,{video_base64}"},
        {"type": "text", "text": f"Full video: {video_path.name}"}
    ]

# Get video duration for defaults
probe = subprocess.run(
    ["ffprobe", "-v", "quiet", "-print_format", "json", "-show_format", path],
    capture_output=True, text=True
)
duration = float(json.loads(probe.stdout)["format"]["duration"])

start_time = start_time or 0
end_time = end_time or duration
clip_duration = end_time - start_time

# Extract clip
with tempfile.NamedTemporaryFile(suffix=".mp4", delete=False) as tmp:
    tmp_path = tmp.name

try:
    subprocess.run([
        "ffmpeg", "-y", "-ss", str(start_time), "-i", path,
        "-t", str(clip_duration), "-c:v", "libx264", "-c:a", "aac",
```

KIMI

开放平台



```
], check=True)
```

```
with open(tmp_path, "rb") as f:
```

```
> video_base64 = base64.b64encode(f.read()).decode("utf-8")
```

```
return [
```

```
    {"type": "video_url", "video_url": {"url": f"data:video/mp4;base64,{v
```

```
    {"type": "text", "text": f"Clip from {video_path.name}: {start_time}s
```

```
]
```

```
finally:
```

```
    if os.path.exists(tmp_path):
```

```
        os.unlink(tmp_path)
```

```
client = OpenAI(
```

```
    api_key=os.environ.get("MOONSHOT_API_KEY"),
```

```
    base_url="https://api.moonshot.cn/v1"
```

```
)
```

```
def agent_loop(user_message: str):
```

```
    """Simple agent loop with multimodal tool support."""
```

```
messages = [
```

```
    {"role": "system", "content": "You are a video analysis assistant. Use wat
```

```
    {"role": "user", "content": user_message}
```

```
]
```

```
while True:
```

```
    response = client.chat.completions.create(
```

```
        model="kimi-k2.6",
```

```
        messages=messages,
```

```
        tools=tools,
```

```
        tool_choice="auto"
```

```
)
```

```
    message = response.choices[0].message
```

```
    messages.append(message.model_dump())
```

```
# No tool calls = done
```

```
if not message.tool_calls:
```

KIMI

开放平台



```
# Execute tool calls
for tool_call in message.tool_calls:

    if tool_call.function.name == "watch_video_clip":
        > args = json.loads(tool_call.function.arguments)
        result = watch_video_clip(
            path=args["path"],
            start_time=args.get("start_time"),
            end_time=args.get("end_time")
        )
        # Multimodal tool result
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": result
        })

# Usage
answer = agent_loop("分析 /path/to/test_video.mp4 这个视频的 8-13 秒发生了什么")
print(answer)
```

## 最佳实践

### 支持的格式

图片支持 png、jpeg、webp、gif；视频支持 mp4、mpeg、mov、avi、x-flv、mpg、webm、wmv、3gpp 格式。

### Tokens 计算及费用

图片与视频进行动态token计算，可以通过 [计算token接口](#)，在开始理解前获取包含图片或视频的请求的token消耗。

一般说来，图片分辨率越高，消耗的token越多；视频由若干张关键帧组成，关键帧的数量越多，分辨率越高，则token消耗越多。

>

## 分辨率说明

我们推荐图片分辨率不超过4k (4096\*2160)，视频分辨率不超过2k (2048\*1080)，再高的分辨率只会增加处理时间，也不会对模型理解的效果有提升。

## 上传文件还是base64

由于我们对请求体的整体大小有限制，所以对于非常大的视频，必须使用上传文件的方式使用视觉理解功能。对于需要多次引用的图片或视频，我们推荐使用文件上传的方式使用视觉理解功能。关于上传文件的限制，请参阅 [文件上传](#) 文档。

图片数量限制：Vision 模型没有图片数量限制，但请确保请求的 Body 大小不超过 100M

URL 格式的图片：不支持，目前仅支持使用 base64 编码的图片内容

## 参数变动说明

在 [chat](#) 文档中有一系列参数，但对于 K2.6/K2.5系列模型，其行为会有所不同。

我们建议用户不要手动设置这些字段，而是使用默认值

参数变动列举如下

| 字段         | 是否必须     | 说明                       | 类型     | 取值                                                                                             |
|------------|----------|--------------------------|--------|------------------------------------------------------------------------------------------------|
| max_tokens | optional | 聊天完成时生成的最大 token 数。      | int    | 默认值为32k，即32768                                                                                 |
| thinking   | optional | <b>新增</b> 该参数控制模型是否启用思考。 | object | 默认值为 <code>{"type": "enabled"}</code> 。只<br><code>{"type":</code><br><code>"enabled"}</code> 或 |

## KIMI 开放平台

|                   |          |                 |       |                                                     |
|-------------------|----------|-----------------|-------|-----------------------------------------------------|
| temperature       | optional | 使用什么采样温度。       | float | k2.6/k2.5 系列模型使用确定值 1.0, 非模式下将使用默认值 0.6。若指定其他值将会报错。 |
| top_p             | optional | 采样方法。           | float | k2.6/k2.5 系列模型使用确定值 0.95。指定其他值，将会报错。                |
| n                 | optional | 为每条输入消息生成多少个结果。 | int   | k2.6/k2.5 系列模型使用确定值 1。若指定其他值，将会报错。                  |
| presence_penalty  | optional | 存在惩罚。           | float | k2.6/k2.5 系列模型使用固定值 0.0。指定其他值，将会报错。                 |
| frequency_penalty | optional | 频率惩罚。           | float | k2.6/k2.5 系列模型使用确定值 0.0。指定其他值，将会报错。                 |

## Tool Use 参数兼容性

当使用工具时，若thinking设置值为 `{"type": "enabled"}`，请注意，为了确保模型的性能，会有以下约束：

- 为了避免思考内容与指定的 `tool_choice` 冲突，`tool_choice` 只能使用“auto”和“none”（默认值为“auto”），取任何其他值将会报错；
- 在多步工具调用过程中，您必须在将本轮会话中工具调用时assistant message里的 `reasoning_content` 保留在上下文当中，否则会报错；
- 官方内置的 builtin 的联网搜索 `$web_search` 工具暂时与 Kimi K2.6/Kimi K2.5思考模式不兼容，可以选择先关闭思考模式后使用联网搜索工具 `$web_search`。

您可以参考[如何使用思考模型](#)正确使用工具调用。

对于 `kimi-k2.6`，`kimi-k2.5` 模型，提供禁用思考能力的选项，需要在请求体中指定

**KIMI** 开放平台  
`"thinking": {"type": "disabled"} :`



`curl` `python`

```
$ curl https://api.moonshot.cn/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $MOONSHOT_API_KEY" \
-d '{
  "model": "kimi-k2.6",
  "messages": [
    {"role": "user", "content": "你好"}
  ],
  "thinking": {"type": "disabled"}
}'
```

## 模型价格

关于token价格，详见 [模型推理价格说明](#)

## 了解更多

- 使用 Kimi 模型进行基准测试，请参考这篇 [基准测试最佳实践](#)
- Kimi K2.6 的最详细的 API 使用示例请见： [使用 Kimi 视觉模型](#)
- 在这里查看在 [Claude Code, Roo Code, Cline](#)中使用 [Kimi模型](#)的方法
- 在这里查看如何配置使用[思考模型](#)
- 联网搜索是Kimi API官方提供的强大工具之一，在这里查看如何使用[联网搜索](#)，以及其他[官方工具](#)
- 在这里查看全部[模型价格](#)，[充值与限速说明](#)，[联网搜索价格说明](#)

🚀 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

**KIMI** 开放平台



< 开始使用 Kimi API

使用思考模型 >

